

30-Day AI Learning Roadmap

Phase 1 of your 60-Day AI Challenge | For B.Tech IT Students

Goal: Go from "I use ChatGPT sometimes" to "I can build, deploy, and explain AI-powered projects in an internship interview" — in 30 days, ~1-2 hours/day.

By the end, you'll have **3 working mini-projects + 1 capstone project on GitHub**, a portfolio-ready resume line, and real prompt-engineering + Python + API skills.

Day 0 Setup Checklist (do this before Day 1)

Tool	Why	Link
VS Code	Your coding editor	code.visualstudio.com
Python 3.10+	Core language for all projects	python.org
Git + GitHub account	Version control, portfolio	github.com
Claude.ai account	Your AI mentor + pair programmer	claude.ai
ChatGPT account (free tier)	Compare outputs, second opinion	chat.openai.com
Anthropic Console / OpenAI Platform	For API keys (used Week 2+)	console.anthropic.com

Create one GitHub repo today called `30-day-ai-challenge`. Every project this month gets a folder inside it.

Week 1 (Days 1-7): AI Landscape + Prompt Engineering Foundations

Milestone: You understand how LLMs work at a practical level, can write structured, reliable prompts, and have your dev environment ready.

Day	Focus	Task (60-90 min)	Resource
1	Orientation	Map the AI tool landscape: chat assistants (Claude, ChatGPT, Gemini), coding assistants (Copilot, Cursor), and "agentic" tools. Write a 1-page note in your own words on what an LLM is and isn't. Install VS Code, Python, Git.	Anthropic's "What is Claude" docs; 3Blue1Brown's "But what is a GPT?" video
2	Prompting basics	Learn zero-shot vs few-shot vs role prompting. Practice: write 5 prompts for the same task (e.g., "explain recursion") in 5 different styles and compare outputs.	docs.claude.com/prompt-engineering/overview
3	Structured prompting	Learn chain-of-thought ("think step by step"), XML/section tags, and requesting JSON output. Practice: ask an AI tool to extract structured data (name, date, amount) from a messy paragraph as JSON.	Anthropic prompt engineering guide (XML tags section)
4	Python refresher	Functions, loops, dictionaries, list comprehensions, reading/writing files. If new to Python, do this as guided practice with an AI tool checking your code.	Real Python "Python Basics"; CodeWithHarry Python playlist
5	Git & GitHub	Learn <code>git init</code> , <code>add</code> , <code>commit</code> , <code>push</code> , <code>.gitignore</code> . Push your Day 1-4 notes/code to your repo. Write your first README.	"Git and GitHub for Beginners" (freeCodeCamp YouTube)
6	APIs 101	Understand what a REST API and an API key are. Get a free API key (Anthropic or OpenAI). Make your first API call from Python that sends a prompt and prints the response.	docs.claude.com → "Getting started" / OpenAI quickstart docs
7	Project 1 + Review	Build a CLI Text Summarizer : paste any article text, script sends it to the API with a well-engineered prompt, prints a 5-bullet summary. Push to GitHub with a README explaining your prompt design choices.	—

Week 1 Project: CLI Text Summarizer (Python + LLM API)

Week 2 (Days 8-14): Programming with AI APIs + Automation

Milestone: You can build small Python tools that talk to an AI API, handle errors, and manage secrets properly — core "AI engineer" muscle memory.

Day	Focus	Task	Resource
8	Python: data handling	Practice nested dictionaries/lists and JSON parsing (<code>json.loads()</code> / <code>json.dumps()</code>) — you'll need this for every API response.	Real Python "Working with JSON"
9	Secure API usage	Learn <code>python-dotenv</code> to keep API keys out of your code (<code>.env</code> files). Refactor Day 6's script to use environment variables.	<code>python-dotenv</code> PyPI docs
10	Conversational loop	Build a simple command-line chatbot that keeps conversation history (a list of messages) and sends it back each turn so the AI "remembers" context.	Anthropic Messages API docs
11	AI productivity tools	Explore Notion AI or Claude for drafting study notes/emails. Use an AI tool to turn your messy class notes into a structured study sheet — this is a real productivity skill, not just a project.	—
12	Embeddings (concept only)	Learn conceptually what embeddings/vectors are and why they power search & "AI that reads your documents" (RAG). No coding needed — just understand the idea with a diagram.	"Embeddings explained simply" — search YouTube
13	Project 2	Build an AI Q&A tool over a text file : load a <code>.txt</code> file (e.g., your class notes), and let the user ask questions; the script feeds the file content + question into the prompt and returns an answer.	—
14	Review + polish	Clean up Project 2's code, add error handling (what if the API fails? empty file?), write a proper README, push to GitHub.	—

Week 2 Project: AI Q&A Tool over your own notes (a simplified RAG concept)

Week 3 (Days 15-21): Building & Deploying Real AI Web Apps

Milestone: You can wrap your AI logic in a real web interface and deploy it — something you can literally show a recruiter a live link to.

Day	Focus	Task	Resource
15	Streamlit basics	Learn Streamlit: it turns a Python script into a web app in ~10 lines. Build a "Hello World" app with a text box and button.	docs.streamlit.io/get-started
16	Connect API + UI	Wire your Week 2 chatbot logic into the Streamlit app — now it's a web-based AI chat app, not just CLI.	—
17	Prompt chaining	Learn how to break a big task into multiple smaller prompts run in sequence (e.g., "outline → expand → polish" for an essay generator). Practice with one example.	Anthropic docs: "Chain prompts"
18	Intro to agent-style tools (light)	Read about what "AI agents" and tool-use mean (conceptually) — important vocabulary for interviews, even at a beginner level. Don't build a full agent yet, just understand the idea.	Anthropic "Building effective agents" blog post
19	Project 3 (build)	Build an AI Resume/Study Buddy web app : user pastes their resume bullet or a topic, app gives structured feedback or a quiz using prompt chaining (Day 17 skill).	—
20	Polish	Add file upload (PDF/text), loading states, and basic error handling to Project 3.	Streamlit file_uploader docs
21	Deploy	Deploy Project 3 to Streamlit Community Cloud (free). Get a live shareable link. Add it to your README and resume draft.	streamlit.io/cloud

Week 3 Project: Deployed AI Resume/Study Buddy Web App (live link!)

Week 4 (Days 22–30): Capstone Project + Placement Prep

Milestone: One polished capstone project that combines everything, plus a resume, GitHub profile, and talking points ready for internship/placement interviews.

Day	Focus	Task	Resource
22	Plan capstone	Pick ONE idea that solves a real problem you've personally felt (see ideas below). Write a 1-page spec: features, tech stack, "what makes it different."	—
23	Build — core logic	Write the backend Python logic (prompt design, API calls, data handling).	—
24	Build — UI	Build the Streamlit (or basic HTML/JS) interface.	frontend basics if needed
25	Build — features	Add 1-2 "wow" features: file upload, history, export to PDF/text, or multi-step reasoning.	—
26	Debug & refine	Test edge cases (empty input, API errors, long text). Refactor messy code.	—
27	Document & deploy	Write a strong README (problem, demo link, screenshots, tech stack, how to run locally). Deploy it.	—
28	Resume + GitHub profile	Add the capstone + 3 mini-projects to your resume with impact-focused bullet points. Polish your GitHub profile README. Use an AI tool to review and tighten your resume language.	—
29	LinkedIn + interview prep	Update LinkedIn with your projects. Prepare 5 talking points: "what is prompt engineering," "what's an API," "what's RAG," "walk me through your capstone," "a bug you fixed."	—
30	Reflect + plan Days 31-60	Review everything you built. Write down 3 skill gaps to fix in the next 30 days (e.g., LangChain, fine-tuning basics, DSA for interviews, a bigger full-stack AI project).	—

Capstone Project Ideas (pick one on Day 22):

- AI Study Planner — generates a personalized revision schedule from a syllabus PDF
- AI Code Explainer — paste any code snippet, get a plain-English walkthrough + bug suggestions
- AI Interview Coach — asks you mock technical/HR questions and gives feedback on your answers
- Smart Notes Summarizer — upload lecture PDFs/notes, get summaries + auto-generated quiz questions

Final Outcome After Day 30

By Day 30 you will have:

- **4 real projects on GitHub:** CLI summarizer, Q&A tool, deployed web app, and a capstone — each with clean READMEs and (where applicable) a live demo link
 - **Practical skills:** prompt engineering (zero/few-shot, chain-of-thought, structured output), Python for AI (APIs, JSON, error handling), Git/GitHub workflow, Streamlit deployment, and the vocabulary to talk about embeddings, RAG, and agents in an interview
 - **Placement-ready materials:** an updated resume with quantifiable project bullets, a clean GitHub profile, and a polished LinkedIn
 - **A clear plan for Days 31-60**, where you can go deeper into frameworks (LangChain), DSA interview prep, or a more advanced full-stack AI product
-

How to Use This With Me Day-to-Day

Each day, just tell me the day number (e.g., "Day 9") and I'll:

- explain the concept simply with examples
- review your code or prompts
- help debug errors
- suggest how to extend the day's project

If you fall behind, don't restart the count — just pick up where you left off. Consistency over speed.